

LEA.

MODULE 04

CSS Layout: Flexbox and Grid

Moving beyond normal document flow to build robust, modern, two-dimensional interfaces.

COURSE SERIES
WEB DEVELOPMENT

Flexbox: One-Dimensional Layout

The Flexible Box Module (Flexbox) is designed for laying out items in a single direction—either a row or a column.

THE CONTAINER

Layout starts by declaring `display: flex;` on a parent element. This element becomes the **flex container**, establishing a new flex formatting context for its direct children.

THE ITEMS

The direct children of a flex container automatically become **flex items**. They line up along the main axis, adapting their size and spacing based on the rules you set.

```
.container {  
  display: flex;  
  gap: 16px;  
}
```



Flex Direction and Wrapping

Control the main axis (horizontal or vertical) and determine what happens when items run out of space in the container.

`flex-direction: row; (Default)`

1 2 3

`flex-direction: column;`

1

2

3

`flex-direction: row-reverse;`

3 2 1

`flex-wrap: wrap;`

1 2 3 4 5 6 7 8 9 10 11

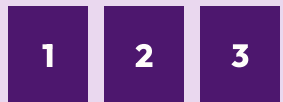
12

Aligning Flex Items

Flexbox provides powerful alignment properties to distribute space along the main axis and align items along the cross axis.

JUSTIFY-CONTENT (MAIN AXIS)

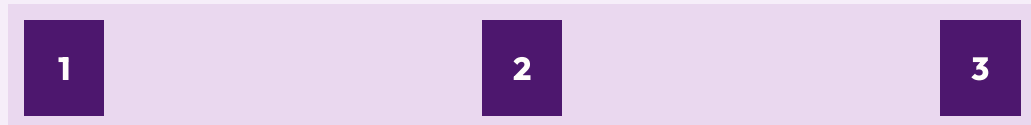
```
justify-content: flex-start;
```



```
justify-content: center;
```



```
justify-content: space-between;
```



ALIGN-ITEMS (CROSS AXIS)

```
align-items: flex-start;
```



```
align-items: center;
```



```
align-items: stretch; (Default)
```



Flex Item Sizing

Control exactly how individual items consume available space or shrink to prevent overflow along the main axis.

flex-basis

The ideal starting size of the item before any available space is distributed or shrinking occurs.

200px

auto

flex-grow

A proportion dictating how much of the **remaining positive space** the item should take up.

grow: 1

grow: 2

flex-shrink

A proportion dictating how much the item should shrink when there is **not enough space**.

shrink: 1

shrink: 2

flex: 1 0 auto;

The Shorthand: Always use the `flex` property instead of the individual properties. It sets grow, shrink, and basis in that exact order.

CSS Grid: Two-Dimensional Layout

While Flexbox handles one dimension at a time, CSS Grid is designed for two-dimensional layouts—managing both rows and columns simultaneously.

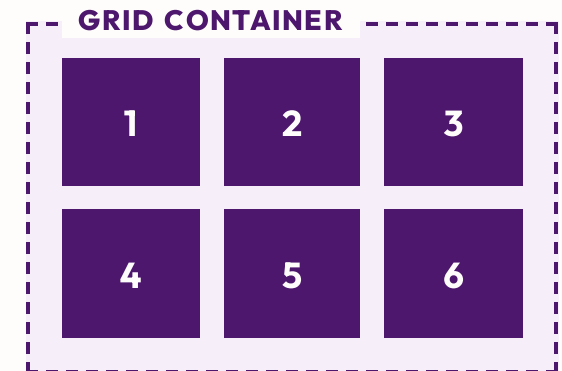
TWO DIMENSIONS

Grid allows you to define a rigid or flexible structure of intersecting horizontal and vertical lines, giving you precise control over where elements are placed.

THE CONTAINER

Like Flexbox, it starts with the parent. Declaring `display: grid`; creates a grid formatting context, turning direct children into grid items that snap into the defined tracks.

```
.container {  
  display: grid;  
  grid-template-  
columns: 1fr 1fr  
1fr;  
  gap: 16px;  
}
```



↓ ROWS

COLUMNS →

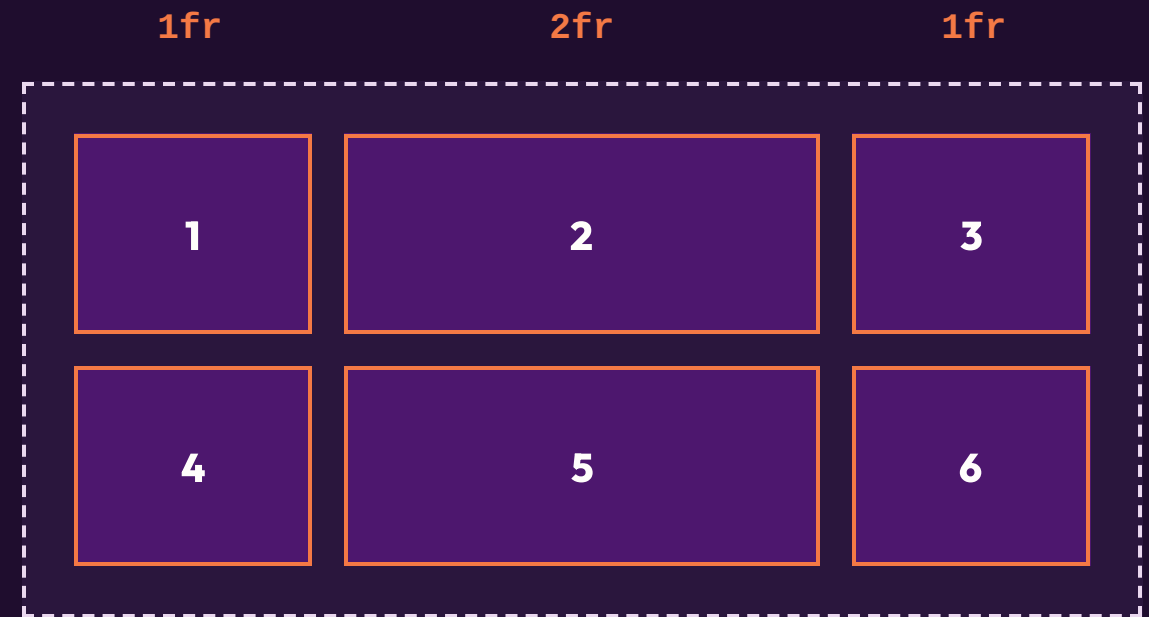
Defining Grid Tracks

You define the structure of your grid by declaring the size of its columns and rows. The `fr` unit makes this incredibly powerful.

THE FRACTIONAL UNIT (FR)

The `fr` unit represents a fraction of the available space in the grid container. It automatically calculates percentages and subtracts gaps for you.

```
.grid-container {  
  display: grid;  
  /* 3 columns: 25%, 50%, 25% */  
  grid-template-columns: 1fr 2fr 1fr;  
  gap: 16px;  
}
```



Placing Grid Items

Position items precisely by referencing the numbered lines that make up the grid, allowing elements to span multiple rows or columns.

GRID LINES

When you define tracks, the browser automatically numbers the lines between them. A 3-column grid has 4 vertical lines (1, 2, 3, 4).

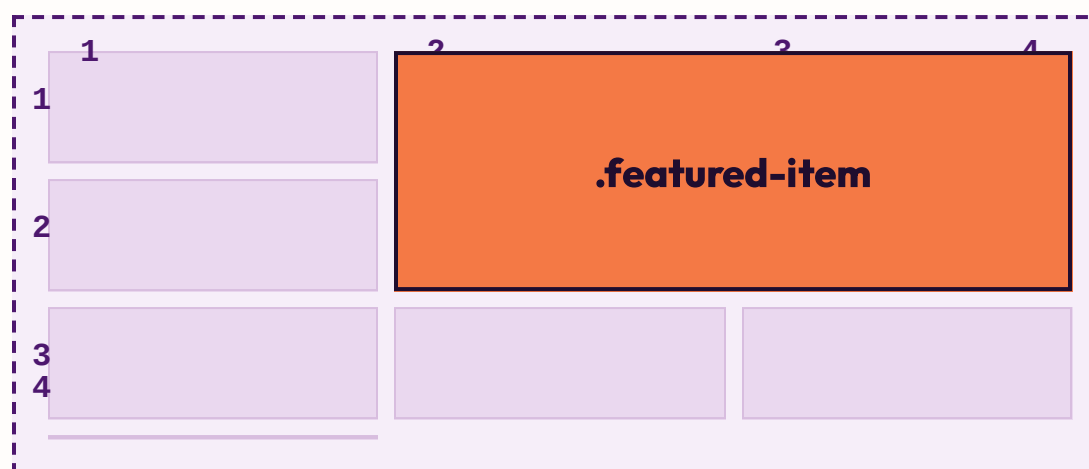
SPANNING COLUMNS

Use `grid-column: start / end;` to tell an item where to begin and where to stop. The item will stretch across the tracks between those lines.

SPANNING ROWS

Similarly, `grid-row: start / end;` controls vertical placement. You can combine both to create large, featured areas in your layout.

```
.featured-item {  
  grid-column: 2 / 4;  
  grid-row: 1 / 3;  
}
```



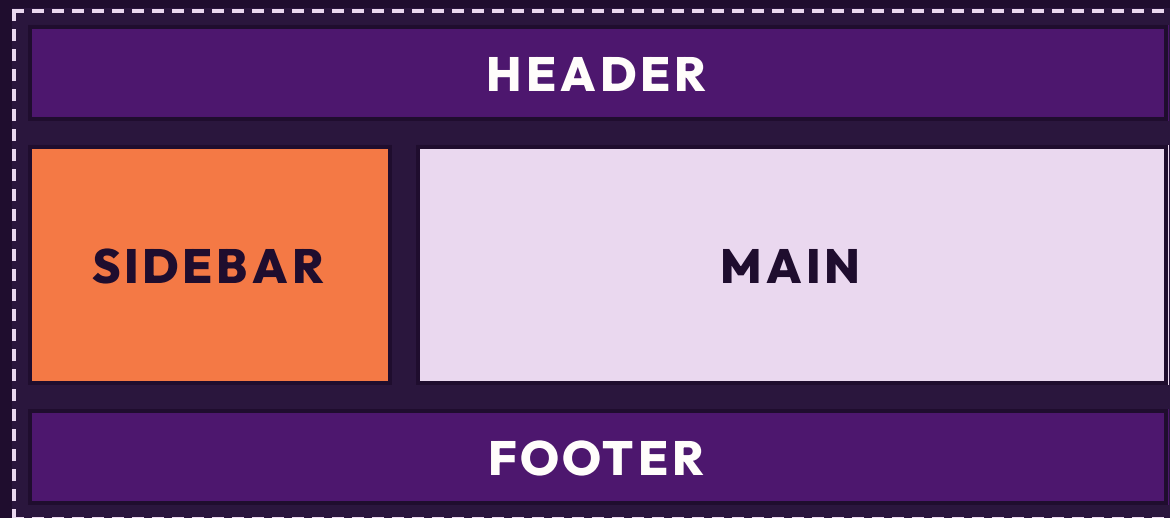
Grid Template Areas

Build semantic, highly readable layouts by naming areas of your grid and assigning elements to those names.

VISUAL LAYOUT IN CSS

The `grid-template-areas` property lets you draw your layout using strings. It acts like ASCII art in your CSS, making complex layouts easier to understand.

```
.layout {  
  display: grid;  
  grid-template-areas:  
    "header header header"  
    "sidebar main main"  
    "footer footer footer";  
}  
  
.header { grid-area: header; }
```



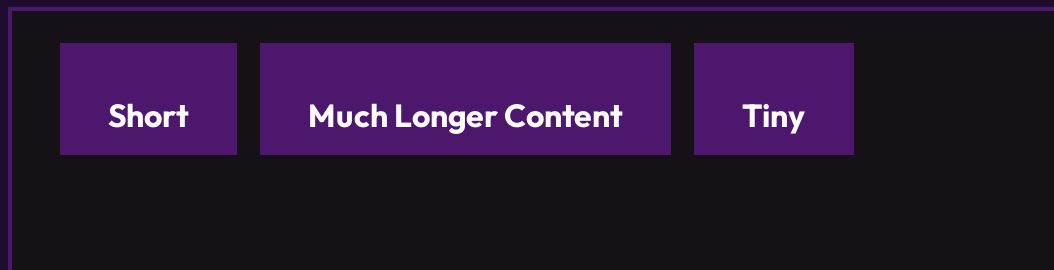
Flexbox vs. Grid

Both are powerful, but they solve different problems. The key difference lies in what dictates the layout: the content or the container.

FLEXBOX

Content-Driven (1D)

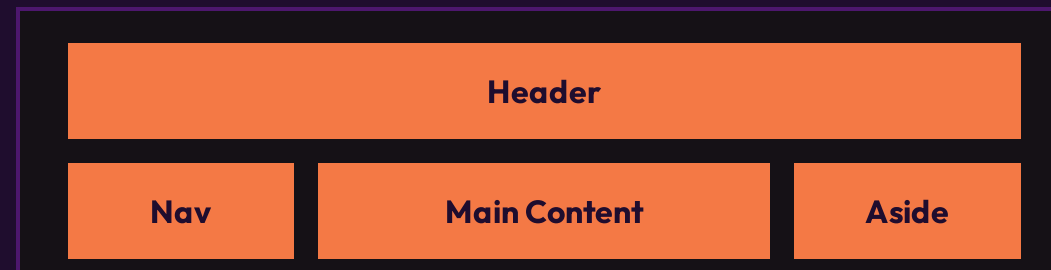
- Works in one dimension at a time (row OR column).
- The size of the items dictates how the layout behaves.
- Great for aligning elements and UI controls.



CSS GRID

Layout-Driven (2D)

- Works in two dimensions simultaneously (rows AND columns).
- The container's grid tracks dictate where items are placed.
- Great for page structure, galleries, and overlaps.



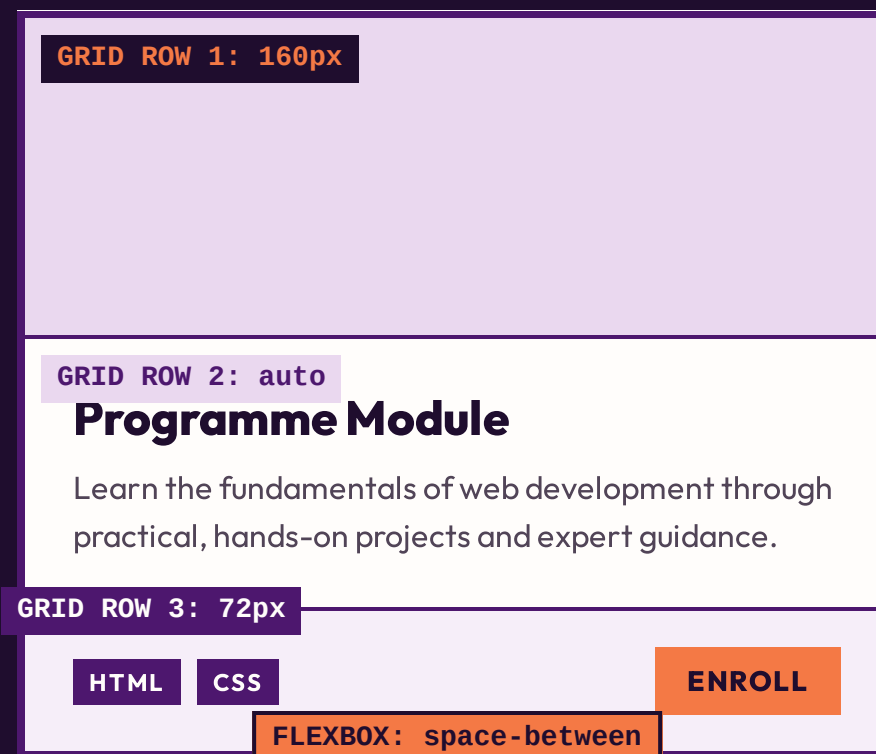
Combining Flexbox and Grid

In the real world, these systems work together. Use Grid for the macro-layout (the skeleton) and Flexbox for the micro-layout (aligning elements inside).

THE PERFECT PARTNERSHIP

A common pattern is a card component: Grid handles the rigid vertical stacking of the image, body, and footer, while Flexbox perfectly aligns the tags and buttons inside the footer.

```
/* Card: Grid rows */  
.card { display: grid; grid-template-rows:  
160px auto 72px; }  
/* Footer: Flexbox */  
.card-footer { display: flex; justify-content:  
space-between; align-items: center; }
```



Lab: Build a Dashboard Layout

Put your layout skills to the test. Build a responsive dashboard using CSS Grid for structure and Flexbox for alignment.

DELIVERABLES

- **Macro Layout:** Persistent sidebar + main area via CSS Grid.
- **Header Nav:** Use Flexbox to align logo, search, profile.
- **Stats:** Responsive summary cards with CSS Grid.
- **Activity:** Feed list aligned with Flexbox.

SUCCESS CRITERIA

- Zero floats and **zero absolute positioning**.
- Adapts cleanly across screen sizes.
- Organized CSS using Grid vs Flexbox appropriately.
- Semantic, accessible HTML structure.